Link and Physical Layers, Security, Anonymity

Exercises 4

Computer Networks and Distributed Programming

What this pack is, and how to use it

- Thirty exercises on the link and physical layers, security and anonymity — and, at the end, the whole stack at once, observed on the chat you have been writing.
- They are for home, numbered E4.1 to E4.30: bring the number to class if one of them resists.
- Most are on paper. Three ask you to capture on your own machine, and the last two to run your own chat.
- There are no solutions. Where an exercise gives a code, a key or a table, it gives everything you need: nothing refers to a figure you do not have.

Byte stuffing

E4.1: Frames are delimited by a `FLAG` byte, and `ESC` escapes a `FLAG` or an `ESC` in the payload, as in the course.

(a) Stuff the payload `A FLAG B ESC C ESC FLAG D`.

(b) Between two flags the receiver finds `A ESC ESC B ESC FLAG C ESC ESC ESC FLAG D`. What payload does it deliver?

(c) A bit error turns one `ESC` of (b) into a `FLAG`. What does the receiver lose, and from where does it start again?

Bit stuffing

E4.2: The flag is `01111110`.

(a) Stuff the data `0111111011111100111110`.

(b) The receiver gets `011111101011111010111110111001111110`. Find the frame, and destuff its payload.

(c) Why does one 0 after five 1s suffice to keep the flag unique?

(d) What should the receiver conclude if it ever sees seven 1s in a row?

Parity and CRC

E4.3 The information in a packet is `1110 0110 1001 0101`, and even parity is used. Give the parity bits of a two-dimensional scheme, with a check field as short as possible.

E4.4 The generator is $G = 10011$.

(a) For $D = 1010101010$, find the CRC bits R. Write $G(x)$ and $D(x)$ as polynomials, and check that $M(x) = x^r D(x) - R(x)$ is divisible by $G(x)$.

(b) Find R for $D = 1000100101$, 0101101010 and 0110100011.

(c) With $G = 1001$, does every single-bit error give a non-zero remainder? Every odd number of errors? Why?

Hamming distance

E4.5: A code uses only some of the possible words, the codewords.

(a) A code has four codewords: `0000000000`, `0000011111`, `1111100000`, `1111111111`. What is its Hamming distance? How many bit errors can it always detect, and how many correct? The receiver gets `0100011101`: which codeword was sent, assuming the fewest errors?

(b) Answer the same questions for the code `00000`, `01011`, `10101`, `11110`, and decode `11011`, `11101` and `01110`.

(c) A code must correct every single error and detect every double one. What is the smallest Hamming distance it can have?

How good ALOHA can be

E4.6: N active nodes share a slotted ALOHA channel; in every slot each transmits with probability p. The efficiency — the fraction of slots that carry exactly one frame — is $Np(1-p)^{N-1}$.

(a) Find the p that maximizes it for a given N.

(b) With that p, find the efficiency as $N \to \infty$. Hint: $(1 - 1/N)^N \to 1/e$.

(c) In pure ALOHA the exponent becomes $2(N-1)$. Explain why, and show that the maximum efficiency is then $1/(2e)$.

(d) In the course the same results came from $S = Ge^{-2G}$ and $S = Ge^{-G}$. What does G correspond to in this model?

Binary exponential backoff

E4.7: In CSMA/CD, after the n-th collision on the same frame a node picks K at random among $0, 1, \ldots, 2^n - 1$ and waits K times 512 bit times.

(a) After the fifth collision, what is the probability that a node picks $K = 4$? How long does it then wait on a 10 Mbps Ethernet?

(b) For $K = 100$, how long does it wait at 100 Mbps? At 1 Gbps, keeping 512 bit times?

The collision that goes unnoticed

E4.8: A and B share a 10 Mbps channel with CSMA/CD; the propagation delay between them is 325 bit times. A starts a frame, and B starts one before A has finished. A frame is at least 512 bits plus 64 bits of preamble, so in the worst case A stops transmitting at 576 bit times.

(a) In the worst case, when does B's signal reach A? Can A finish before it detects B?

(b) Express τ and 2τ in bit times, and say what the answer means for the minimum frame length.

Hidden and exposed

E4.9: Five radio stations stand on a line: A at 0 m, B at 80 m, C at 160 m, D at 240 m, E at 320 m. Each reaches every station within 100 m, and no further.

(a) A sends to B while C also sends to B. What happens at B? Would listening to the channel before transmitting have prevented it?

(b) B is sending to A, and C wants to send to D. What does carrier sense make C do? Was it necessary?

(c) Now A sends to B with RTS/CTS. Who hears the RTS, who hears the CTS, and who must stay silent while the data flows? May C send to D meanwhile? May D send to E?

Code division, decoded

E4.10: Four stations have these chip sequences; a station sends its sequence for a 1, its negation for a 0, nothing when silent. The receiver takes the inner product with a station's sequence and divides it by 8.

A	-1	-1	-1	$+1$	$+1$	-1	$+1$	$+1$
B	-1	-1	$+1$	-1	$+1$	$+1$	$+1$	-1
C	-1	$+1$	-1	$+1$	$+1$	$+1$	-1	-1
D	-1	$+1$	-1	-1	-1	-1	$+1$	-1

(a) Check that A and B are orthogonal, and that A with itself gives 1.

(b) The receiver gets $(-1, +1, -3, +1, -1, -3, +1, +1)$. Which stations transmitted, and which bit did each send?

(c) What arrives if C sends a 1 and D a 0, and the others are silent?

Addresses, hop by hop

E4.11: Three LANs are joined by two routers. Subnet 1 holds hosts A and B and one interface of router R1; subnet 2 holds hosts C and D, the other interface of R1 and one interface of router R2; subnet 3 holds hosts E and F and the other interface of R2. Use `192.168.1.0/24`, `192.168.2.0/24` and `192.168.3.0/24`.

(a) Assign an IP address and a MAC address to every interface of A, both routers, and F.

(b) A sends an IP datagram to F. Give the source and destination MAC addresses of the frame, and the source and destination IP addresses of the datagram, (i) from A to R1, (ii) from R1 to R2, (iii) from R2 to F.

(c) Which ARP queries are needed if every ARP table starts empty, and who sends each one?

Switches and hubs

E4.12 Six nodes, A to F, are star-connected to a switch whose table starts empty. In order: B sends to E, E replies to B, A sends to B, B replies to A. After each event, show the switch table and the links on which the frame is sent; justify each.

E4.13 A core switch joins three departmental switches, a Web server, a mail server and a router to the Internet; each departmental switch has three hosts. Every link is 1 Gbps.

(a) What is the largest total throughput among the nine hosts and the two servers?

(b) And if the three departmental switches become hubs?

(c) And if every switch becomes a hub?

VLANs

E4.14: Two questions on 802.1Q.

(a) How many VLANs can a switch that supports 802.1Q configure at most? What sets the limit?

(b) N switches, each supporting the same K VLAN groups, are to be connected. How many ports are needed without trunking, and how many with it? Justify.

ARP, seen from ping

E4.15: Capture with display filter `arp || icmp` while you run, one at a time:

```
$ ip neigh ; sudo ip neigh flush all
$ ping -c 1 <your gateway>
$ ping -c 1 <an address of your subnet that nobody uses>      then:  ip neigh
$ ping -c 1 8.8.8.8
```

(a) For the gateway: which frames, in which order? Which fields of the ARP request are still unknown, and to whom is it addressed?

(b) For the unused address: what goes on the wire, how many times? What does `ping` print, which machine generated that message, and what state does `ip neigh` show?

(c) For 8.8.8.8: compare the destination MAC of your request with the gateway's entry in `ip neigh`. Was 8.8.8.8 ever asked for?

The limits of a telephone line

E4.16: A telephone channel carries frequencies from 300 to 3400 Hz, with a signal-to-noise ratio of 30 dB.

(a) What is its bandwidth in hertz? What is S/N as a pure ratio?

(b) What is its Shannon capacity?

(c) What does Nyquist allow on it with 2 signal levels? With 16?

(d) A modem sends 2400 symbols per second with QAM-16. What is its bit rate? And with 64 points in the constellation?

(e) Compare (b) with the 33.6 kb/s at which modems stopped. What would it take to go faster on the same line?

Line codes

E4.17: Take the bits `1 0 1 1 0 0 0 1`.

(a) Draw them in NRZ; NRZI, where a 1 is a transition and a 0 is none; Manchester, with a 1 as a low-to-high transition in the middle of the bit and a 0 as high-to-low; AMI, where a 0 is zero and the 1s alternate, the first one positive.

(b) Which of the four still gives the receiver a transition during a run of eight 0s? During a run of eight 1s?

(c) In 4B/5B, `0000` becomes `11110` and `0001` becomes `01001`. Encode `0000 0001 0000`. What is the longest run of 0s on the line? What bit rate does the line need to carry 100 Mbit/s of data?

Keys, substitution and transposition

E4.18 N people each want to talk privately with each of the other $N - 1$; everything any two say is visible to all. How many keys does the system need with symmetric cryptography? And with public keys?

E4.19 A substitution cipher maps the plaintext alphabet `abcdefghijklmnopqrstuvwxyz` to `QWERTYUIOPASDFGHJKLZXCVBNM`.

(a) Encrypt `this is an easy problem`; decrypt `ATNL DXLZ LZQN LTEKTZ`.

(b) With the key `MEGABUCK`, write `meetmeatthestationatnoon` in rows of eight under the key, and read the columns in the alphabetical order of the key's letters. What is the ciphertext?

(c) Given only a ciphertext, how can you tell which of the two methods produced it?

A 3-bit block cipher, with and without CBC

E4.20: A block cipher works on 3 bits with this table:

Input	000	001	010	011	100	101	110	111
Output	110	111	101	100	011	010	000	001

The plaintext is `100100100`.

(a) Without CBC, what is the ciphertext?

(b) Trudy sniffs it. She knows a 3-bit block cipher is used without CBC, but not the table. What can she conclude?

(c) With CBC and $IV = 111$, what is the ciphertext? What must the receiver know to decrypt it?

RSA with small numbers

E4.21: RSA with $p = 5$ and $q = 11$.

(a) What are n and z?

(b) Why is $e = 3$ an acceptable choice?

(c) Find a d with $de \equiv 1 \pmod{z}$ and $d < 160$.

(d) Encrypt $m = 8$, then decrypt the result. Use $[(a \bmod n)(b \bmod n)] \bmod n = ab \bmod n$ to keep the numbers small.

Diffie-Hellman with small numbers

E4.22: Diffie-Hellman with $p = 11$ and $g = 2$; Alice picks $S_A = 4$, Bob $S_B = 7$.

(a) Prove that the two keys computed by Alice and Bob are always equal.

(b) Compute Alice's and Bob's public values T_A and T_B.

(c) Compute the shared key.

(d) Draw a time diagram, one line each for Alice, Bob and Trudy, in which Trudy sits in the middle and ends up sharing a key with each of them.

Authenticating both ways

E4.23: Alice authenticates to Bob like this: she sends `I am Alice`; Bob replies with a fresh nonce R; Alice returns $K_{A-B}(R)$, encrypted with the key she shares with Bob; Bob decrypts it and checks that it is R. As seen in the course, this works.

(a) Now Bob must also authenticate himself to Alice, with the same protocol and the same key: he sends `I am Bob`, she replies with a nonce, he returns it encrypted.

(b) The two runs can be interleaved in any order. Show how Trudy, who does not know K_{A-B}, can make Bob believe she is Alice. Hint: make Bob answer his own question.

(c) Propose a change that stops the attack.

TLS: four quick questions

E4.24: Answer with the TLS handshake and records seen in the course.

(a) True or false: a TLS record has a field for TLS sequence numbers. Where does the sequence number go, then?

(b) What are the random nonces in the TLS handshake for?

(c) A TLS session uses a block cipher with CBC. True or false: the server sends the IV to the client in the clear.

(d) Bob opens a TCP connection to Trudy, who pretends to be Alice and sends Bob Alice's certificate. At which step of the handshake does Bob discover that he is not talking to Alice?

A TLS handshake you can read

E4.25: Capture with display filter `tls`:

```
$ curl -s -o /dev/null --tls-max 1.2 https://www.unina.it
$ curl -s -o /dev/null https://www.unina.it
```

The first forces TLS 1.2, whose handshake the course followed; the second allows 1.3, so that you can see what the newer version hides. The server may answer `403` to `curl`: it does not matter here, the handshake happens before the server sees the request at all.

(a) In the 1.2 capture, list the handshake messages in order. Where are the two random nonces?

(b) Which site you are visiting is readable in the clear: in which message, which field?

(c) Open the certificate: subject, issuer, validity. Who vouches for whom?

(d) From which record on is everything encrypted?

(e) In the 1.3 capture, which of these can you still read? What has a network observer lost?

A stateful firewall

E4.26: The internal network is `222.22.0.0/16`. Write the filter table and the connection table of a stateful firewall, as restrictive as possible, that:

(a) lets every internal user open Telnet sessions to external hosts;

(b) lets external users browse the company Web site at `222.22.0.12`;

(c) blocks every other inbound and outbound packet.

The connection table currently holds three connections, all opened from inside. Invent the addresses and ports you need, and say for each row of the filter table whether it consults the connection table.

Who knows what in a circuit

E4.27: Alice reaches the Web site S through Tor, over a circuit with a guard G, a middle relay M and an exit X.

(a) For each of G, M, X and S, say whether it knows Alice's IP address, the name of S, and the content of the request. Answer twice: when Alice reaches S over `http://` and over `https://`.

(b) An observer sees both the traffic between Alice and G and the traffic between X and S, but cannot decrypt anything. What can she still learn, and how?

(c) Why three relays? What would one relay give away, and what would two?

The whole stack, on paper

E4.28: You walk into a room, plug your laptop into the Ethernet, and download a Web page. Nothing is cached: no address, no ARP entry, no DNS answer.

(a) List every protocol step from the moment the laptop is switched on until the page is displayed: Ethernet, DHCP, ARP, DNS, TCP, HTTP.

(b) Say explicitly how the laptop learns the IP address of its gateway, and then its MAC address.

(c) For every step, which addresses are known and which are still unknown?

Your chat, the whole stack (1 of 2)

E4.29: Server on one machine, client on another of the same LAN; capture on the client's real interface, with no filter. On the client, in order:

```
$ nmcli connection down <conn> ; nmcli connection up <conn>    own machine only
$ sudo ip neigh flush all ; resolvectl flush-caches
$ ./chat <server name or address> 5000                         one short message, then quit
```

Renew the address only on your own machine: in a lab, skip that line and start from the second. Connect by name if the server has one in your DNS; otherwise by address, and the DNS part is skipped.

(a) Build the timeline: every protocol that appears, in order, with the number of its first packet. How many DHCP messages do you see, and why maybe not four?

(b) For each ARP and DNS exchange: who asked, about whom, and why at that moment?

Your chat, the whole stack (2 of 2)

(c) Take the frame that carries your short message. Layer by layer, give the length of each header and the field that says what comes next: EtherType, IP protocol, destination port — and your own type byte. Check that the lengths add up to the frame length.

(d) Send one message with a 3000-byte body, first with the connected version of your chat, then with the datagram one; raise your declared maximum length for the test if you must. How many frames does each produce? Which layer cut the message, and how does the receiver put it back together?

(e) In the datagram version, what happens to the whole message if one of those frames is lost? And in the connected one?

(f) Draw the complete time diagram of (a), one vertical line per machine involved: client, server, gateway, DHCP and DNS servers.

What your chat costs under load (1 of 2)

E4.30: Take the concurrent versions of your chat — with processes, with threads, event-driven, and with a pool — and measure them under the same load.

- The load: write a client that starts N connections; each sends one message every 100 ms, carrying the time it was sent, and records how long each message takes to reach another client. Run it for 60 seconds.
- The measures: for $N = 10$, 50 and 200: average and 95th percentile of the delay; messages delivered per second; the server's CPU time and resident memory (`ps -o rss,time`).
- Put everything in one table: one row per version, one group of columns per N.

What your chat costs under load (2 of 2)

(a) Which version degrades first as N grows, and what is its bottleneck: CPU, a lock, one queue per client, the number of threads? Show it from your numbers, not from the lectures.

(b) During a run with $N = 50$, suspend one client with `Ctrl-Z` for ten seconds. Which versions slow everybody down, and why those?

(c) Run server and clients on two machines. At which N does the network become the bottleneck, and how do you tell from the measures? Compare the delivered rate with the link rate.

(d) With 200 clients, how many bytes of your messages are headers of TCP, IP and Ethernet? Would the datagram version change that?

Tools you need

- Wireshark: as in the previous packs, capturing on the interface you really use: loopback shows no Ethernet, no ARP and no DHCP.
- Terminal: `ip`, `ping`, `curl`, `nmcli`, `resolvectl`. Commands with `sudo` change your machine's state: run them where you are allowed to.

```
$ sudo apt install iproute2 iputils-ping curl
```

- E4.29 and E4.30: need two machines on the same LAN and your own chat, in the versions you have written. A virtual machine in bridged mode counts as a second machine.